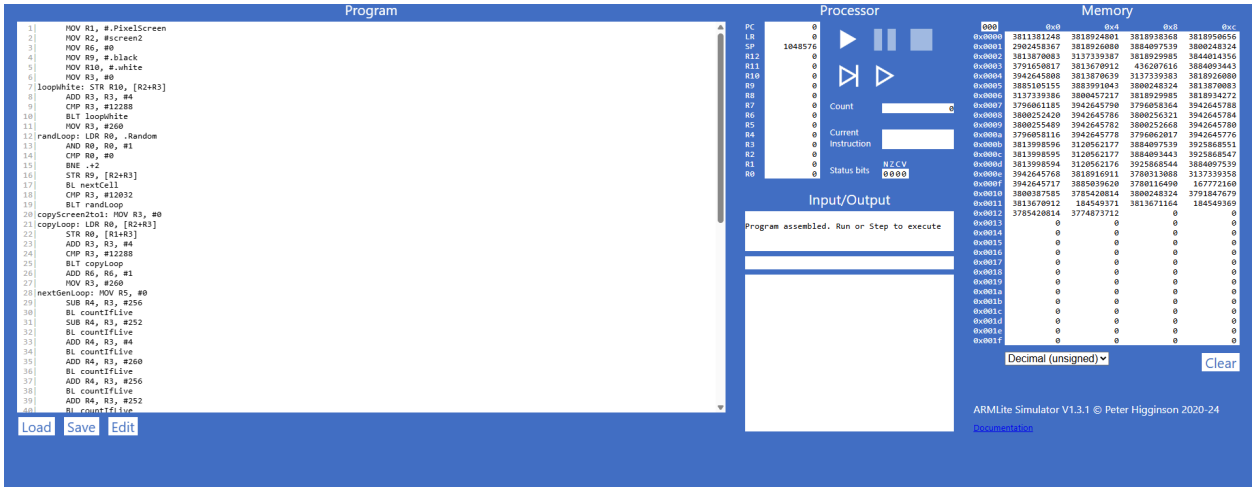
7.1.1
0x00000065 because 101 in decimal is 0x65 in hex
7.1.2
0x00000101 because 0x101 represents a hexadecimal in ARMlite
7.1.3
0x00000005 because 0b means binary, and 101 in binary is 5
7.1.4
No, because those are memory addresses, which are fixed

7.2.1
Because ARMlite uses 32 bit architecture, which contains 4 bytes for each memory word

7.3.1



7.3.2
Memory values have moved around, added and subtracted from each other

7.3.3
It represents the memory value addressed to each register

7.4.1
The highlighting is the instruction that the program is currently paused on

7.4.2
Skips to the next line, or instruction

7.4.3
It paused just before the breakpoint

7.5.1
It will add 8 to r0 and store the value (9) in r1

7.5.2

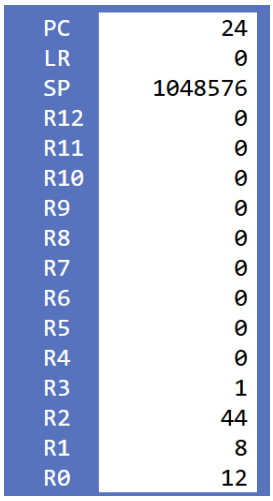
PC	16
LR	0
SP	1048576
R12	0
R11	0
R10	0
R9	0
R8	0
R7	0
R6	0
R5	0
R4	0
R3	84
R2	109
R1	9
R0	1

7.5.3
Answer at R5

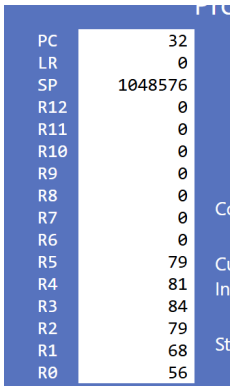
PC	40
LR	0
SP	1048576
R12	0
R11	0
R10	0
R9	0
R8	0
R7	0
R6	0
R5	294
R4	312
R3	376
R2	397
R1	392
R0	300

7.5.4

Instruction	Decimal value of the destination register after executing this instruction	Binary value of the destination register after executing this instruction
MOV R0, #12	12	1100
AND R1, R0, #10	8	1000
ORR R2, R1, #3	11	1011
EOR R3, R2, R1	3	0011
LSL R2, R2, #2	44	101100
LSR R3, R3, #1	1	0001
HALT		



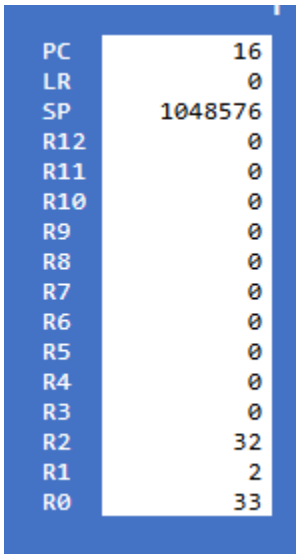
7.5.5



MOV R0, #7
LSL R0, R0, #3
ADD R1, R0, #12
ADD R2, R1, #11
ADD R3, R2, #5
SUB R4, R3, #3
SUB R5, R4, #2
HALT

7.5.6

MOV R0, #33
SUB R1, R0, #31
LSL R2, R1, #4
HALT



7.6.1
Because after LSL #18, the first bit has become 1, signifying a negative number in two's complement

7.6.3
1 : 0b00000000000000000000000000000001
-1 : 0b11111111111111111111111111111111
2 : 0b00000000000000000000000000000010
-2 : 0b11111111111111111111111111111110

Negative numbers contain a lot more 1

7.6.4

```
1       MOV R0, #5
2       MVN R1, R0
3       ADD R1, R1, #1
4       HALT
```

R2	0
R1	-5
R0	5